A Representation-Independent Implication Core for K215

Checked proof reductions, certificate refinements,
and a bounded search below 215 states

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026

Abstract

The main result is a stronger and smaller replacement for the dominated-ρ_8 completeness proof used by K215. The new theorem applies to an arbitrary implication operation and a reflexive relation closed under placing a related term inside either operand. Restricted ρ_8 instances and modus ponens suffice: no distinguished true numeral, least element, numerical encoding, injectivity, decoding, or identity $0 \Rightarrow 0 = 0$ is needed. A compressed finite derivation retains 84 of the original 117 numbered helper schemas; the main unrestricted-ρ_8 theorem depends on 82. A direct composition/exchange argument replaces the local deduction calculus. The generic result is checked in Lean and Rocq, and a numerical Lean adapter preserves the current public interface. A further checked specialization shows that four explicit implication-context schemes suffice, with no order relation at all. A numerical client proves equality with the existing $C222$ calculus for every code.

A second checked contribution composes the incoming 1,426-entry normalized source certificate with balanced lookup, using an exact refinement theorem rather than a new semantic simulation proof. An optional Lean patch makes the register-to-tape halting bridge an instance of an existing positive-simulation theorem. We compare alternative 215-state machines by proof obligation, prove a logical obstruction to one restricted-vacuity recovery strategy, and search a different family of state eliminations. No verified 214-state machine is produced.

Contents

1 Outcome and exact baseline **3**
1.1 Prior work included in the review . . . 3
1.2 Evidence levels . . . 4

2 A stronger dominated-implication theorem **4**
2.1 Eliminating the initial theorem assumption . . . 5
2.2 Compressing the finite bootstrap . . . 5
2.3 A direct final argument replaces deduction . . . 5
2.4 A syntactic corollary . . . 6
2.5 Four explicit implication schemes suffice . . . 6

3 The numerical adapter and verification **7**

4 Normalized certificates with balanced lookup **8**
4.1 Exact refinement replaces a new semantic proof . . . 8
4.2 Operation counts are not a speedup claim . . . 8

5 A reusable halting bridge: benefit and cost **9**

6 Which other 215-state candidate is easier? **9**

7 Searching for 214 states **10**
7.1 A concrete FT215 follow-up . . . 10
7.2 Why removing truth does not automatically remove a boot state . . . 11

8 A logical obstruction to one recovery strategy **11**

9 Integration and reproduction **12**
9.1 Remaining limits . . . 12

A Trust and provenance **13**

1 Outcome and exact baseline

The most useful immediate change is to replace the numerical dominated-implication derivation by an abstract proof core and a thin numerical adapter. This removes assumptions from the mathematical theorem as well as source from the maintained proof. It does not change the Turing-machine table.

The baseline is repository commit `aa66d0a33f1f8237a348899eb8095b045a502832`, retrieved through the GitHub connector. The public K215 readme records a 215-working-state deterministic binary machine with a separate halt and initialization included. Its convention is one initially blank two-way infinite tape and left/right moves only. The accepted dense table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

The mathematical target remains

$$\mathrm{ZFC} \vdash \neg\,\mathrm{Halt}(K215, \mathrm{blank}) \iff \mathrm{Con}(\mathrm{ZFC}).$$

The existing external Lean and Rocq theorems connect blank-start halting to a finite syntactic ZFC refutation. They do not assert unconditional nonhalting or an internal PA endpoint. The maintained Lean phase-five checkpoint records a 184-module endpoint closure, with exact bindings for reused predecessor objects [1, 2]. These baseline facts are repository records; the new focused checks below are separately documented.

1.1 Prior work included in the review

The review includes the three ZIP files added at the baseline commit, not merely the maintained proof subtree:

- `Affine_Proof_Cores_and_Transport_2026-10-21.zip`;
- `K215_Certificate_Refinements_2026-10-21.zip`;
- `K215_Checked_Simplifications_2026-10-21.zip`.

These packets already contain or discuss ordinary zero-offset certificates, the 1,426-entry normalized certificate, balanced lookup for an earlier 2,240-entry certificate, marker transport, decoder certificates, concrete zero-credit enumeration, and several large restricted searches. Those ideas are not presented here as new discoveries. The new source-checker result is the checked composition of the normalized and balanced refinements. The positive-simulation theorem is also inherited; the new contribution there is its checked integration into the current `KSim` interface.

1.2 Evidence levels

Component	Evidence and precise boundary
Implication cores	Generic and four-scheme theorems checked in Lean and Rocq; unchanged numerical client and exact $C\text{Four} \leftrightarrow C222$ checked in Lean. Full K215 endpoint not rebuilt.
Normalized plus balanced source proof	Exact all-register source equivalence, fresh compilation and recursive `coqchk`. Source layer only.
Positive simulation integration	Current Lean types and axiom reports compared before and after. First-use total source size increases.
Hidden-coordinate obstruction	Lean nonderivability theorem for a specified logical fragment, not the full set-theoretic bundle.
State reduction	Bounded exact-local search and controls. No accepted 214-state table and no state-count lower bound.

2 A stronger dominated-implication theorem

The old proof is phrased over natural-number codes. Its architecture obscures which numerical facts are actually needed. The new derivation separates the implication calculus from its encoding.

Definition 2.1. A *dominance language* consists of a type A, a binary operation $x \Rightarrow y$ on A, and a relation $\preceq$ satisfying

$$a \preceq a, \tag{1}$$
$$u \preceq x \implies u \preceq (x \Rightarrow y), \tag{2}$$
$$u \preceq y \implies u \preceq (x \Rightarrow y). \tag{3}$$

There is no separate transitivity or antisymmetry requirement. Define

$$\rho(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

A predicate $\mathcal{T} \subseteq A$ is closed under modus ponens if

$$\mathcal{T}(x \Rightarrow y) \text{ and } \mathcal{T}(x) \quad \implies \quad \mathcal{T}(y).$$

Theorem 2.2 (Restricted rows suffice)**.** *Let $(A, \Rightarrow, \preceq)$ be a dominance language. Suppose $\mathcal{T}$ is closed under modus ponens and*

$$u \preceq a \quad \implies \quad \mathcal{T}(\rho(a, b, c, u)) \qquad (a, b, c, u \in A).$$

Then every unrestricted row belongs to $\mathcal{T}$:

$$\mathcal{T}(\rho(a, b, c, u)) \qquad (a, b, c, u \in A).$$

The same assumptions derive composition and distribution:

$$\mathcal{T}\big((a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c))\big),$$
$$\mathcal{T}\big((a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c))\big).$$

The conclusion assumes no distinguished element 0, known theorem at that element, least-element property, pairing function, or decoder. The operation need not be injective; different expressions may denote the same element. This directly accommodates codes that need not decode to ordinary formulas.

2.1 Eliminating the initial theorem assumption

An intermediate bootstrap is conveniently proved assuming one theorem $\mathcal{T}(\tau)$. This can be discharged internally. For any argument a of the desired conclusion, choose

$$\tau = \rho(a, a, a, a).$$

Reflexivity gives $a \preceq a$, so the restricted-row premise proves $\mathcal{T}(\tau)$. Instantiate the bootstrap with this τ.

No global nonemptiness assumption is required: the first supplied argument supplies the seed. For an empty carrier the universal statement is vacuous. The private seeded presentation is a proof organization device, not an axiom of the public theorem.

2.2 Compressing the finite bootstrap

The baseline module has 117 numbered helper schemas: 58 bootstrap lemmas and 59 condensed-detachment lemmas. A search for alternative instances of existing schemas produced thirteen new two-premise modus-ponens steps. Replacing those steps and taking the acyclic dependency closure removes 33 numbered schemas. The result retains 84; 82 lie in the dependency closure of the main unrestricted-row result. The two additional schemas preserve the exported distribution theorem.

Each replacement is an explicit proof term. The generator selects substitutions, but Lean or Rocq checks the resulting formulas and modus-ponens applications. The archive contains the pinned input, replacement plan, generated proofs, and compiler receipts. No optimality claim is made for the remaining 84 schemas.

For example, two of the new dependency edges are

$$\texttt{cd75} \longleftarrow (\texttt{b52}, \texttt{cd92}), \qquad \texttt{cd103} \longleftarrow (\texttt{cd104}, \texttt{cd69}).$$

The complete substitutions appear in the delivered source. The graph is topologically sorted before emission; numbered labels retain provenance rather than prescribing file order.

The compressed graph no longer needs the special-zero derivations. Two remaining proofs of the distinguished numeral can use the private seed hypothesis instead. All normalizations using $wim(0, 0) = 0$ then disappear from the abstract derivation. The final generic Lean core imports only `Init`.

2.3 A direct final argument replaces deduction

The earlier final step introduced a local proof syntax, proved its soundness, proved a deduction theorem by induction, and discharged several hypotheses. That machinery proves more than this target requires.

The finite bootstrap supplies

$$\begin{aligned} B'(x, y, z) &= (x \Rightarrow y) \Rightarrow ((y \Rightarrow z) \Rightarrow (x \Rightarrow z)), \\ C(x, y, z) &= (x \Rightarrow (y \Rightarrow z)) \Rightarrow (y \Rightarrow (x \Rightarrow z)), \\ K(x, y) &= x \Rightarrow (y \Rightarrow x), \\ P(x, y) &= ((x \Rightarrow y) \Rightarrow x) \Rightarrow x. \end{aligned}$$

From B' and modus ponens, derived implications compose. From B' and C, an established implication can be transported under a fixed antecedent:

$$\mathcal{T}(x \Rightarrow y) \quad \Longrightarrow \quad \mathcal{T}\big((z \Rightarrow x) \Rightarrow (z \Rightarrow y)\big).$$

To obtain this, exchange the two antecedents of $B'(z, x, y)$ with C, then apply modus ponens with $x \Rightarrow y$.

Set

$$d = c \Rightarrow u, \quad f = d \Rightarrow a, \quad g = a \Rightarrow c, \quad q = d \Rightarrow c.$$

Composition gives $\mathcal{T}(f \Rightarrow (g \Rightarrow q))$. Peirce gives $\mathcal{T}(q \Rightarrow c)$. Transporting Peirce under g and composing gives

$$\mathcal{T}(f \Rightarrow (g \Rightarrow c)).$$

Finally, $K(c, b)$ gives $\mathcal{T}(c \Rightarrow (b \Rightarrow c))$. Transport that implication under g and compose again:

$$\mathcal{T}(f \Rightarrow (g \Rightarrow (b \Rightarrow c))) = \mathcal{T}(\rho(a, b, c, u)).$$

This uses no general deduction theorem and no distribution schema in the final step. The two substantive hypotheses f and g are each used once. Distribution remains exported for compatibility.

2.4 A syntactic corollary

Corollary 2.3 (Subterm restriction)**.** *On the free language of implication trees, let $u \preceq a$ mean that u is a subterm of a. Modus ponens and only those ρ_8 instances for which u is a subterm of a derive every unrestricted ρ_8 instance.*

Proof. Subterm satisfies reflexivity and both operand-insertion rules. Apply Theorem 2.2. □

The delivered Lean example also proves equality of the restricted and full derivability predicates. This is an alternative formulation of the propositional component, not yet a smaller TM implementation: recognizing subterms may cost more than the present numerical restriction.

2.5 Four explicit implication schemes suffice

The actual finite derivation uses only four shapes of witness for the restricted row. This permits a stronger formulation than a relation closed under every implication context.

Theorem 2.4 (Four-scheme completeness)**.** *Let A be any type, $\Rightarrow$ any binary operation, and $\mathcal{T}$ any predicate closed under modus ponens. It suffices to supply the following four families, with all displayed variables arbitrary:*

Family	*Supplied row*	*Position of u in the first argument*
Same	$\rho(u, b, c, u)$	*empty path*
Right	$\rho(x \Rightarrow u, b, c, u)$	R
Left–left	$\rho((u \Rightarrow x) \Rightarrow y, b, c, u)$	LL
Left–left–right	$\rho(((x \Rightarrow u) \Rightarrow y) \Rightarrow z, b, c, u)$	LLR

Then $\mathcal{T}(\rho(a, b, c, u))$ for every a, b, c, u.

No order relation, hereditary closure, numerical comparison, injectivity, truth element or decoder appears in the hypothesis. These are four *schemes*, each with arbitrarily many instances; no minimality claim is made.

The proof specializes the nine guarded leaves of the compressed derivation. The right witness occurs at `b1`; left–left–right at `b31`; left–left at `cd62`, `cd64` and `cd67`; the empty path at `b0`, `cd34`, `cd82` and the seed. Every intervening conditional helper passes its exact witness unchanged. A deterministic generator inventories these uses and emits the checked proof. This theorem does not

follow just by supplying the four-path relation to Theorem 2.2, because that relation lacks the latter theorem's full operand-insertion closure.

The separate numerical calculus CFour keeps all other $C222$ rules, including truth, generalization and the original two bundle guards. Only ρ_8 is replaced by these four families. The delivered Lean file `KCodeFourShape.lean` proves

$$\forall n \in \mathbb{N}, \quad C\text{Four}(n) \longleftrightarrow C222(n), \qquad \neg C\text{Four}(F) \longleftrightarrow \neg C222(F).$$

The first equivalence covers all codes, without a formula-decoding premise. This is stronger than equiconsistency alone. The existing $C222$ calibration therefore transfers by composition.

The abstract theorem passed fresh Lean compilation and a separate direct Rocq port with recursive `coqchk`. The numerical equivalence passed a fresh 13-module Lean build. Its three public axiom reports are empty. The numerical adapter has not been ported to Rocq in this packet.

This supplies a concrete alternative propositional interface for an enumerator. Constructing a small fixed collection of contexts may be cheaper than checking a relation between arbitrary codes, but the extra parameters and larger terms may instead cost states. Only the propositional interface loses arithmetic side conditions; the remaining code calculus keeps its original arithmetic operations and bundle guards. No compiled smaller TM follows automatically.

3 The numerical adapter and verification

For the present coding, use $A = \mathbb{N}$, $\Rightarrow = wim$, and $\preceq = \leq$. Existing pairing inequalities prove operand insertion. The adapter retains the public declarations `DomImp`, `le_l`, `le_r`, `sub`, `wim00`, `composition`, `distribution`, and `rho8_all`. The compatibility record retains `truth`, although the new result does not use it.

The stronger interface `rho8_all_of_restricted` assumes only modus ponens and the dominated rows. The original `rho8_all` immediately applies it. Keeping the old declarations avoids a simultaneous client rewrite.

Presentation	Bytes	Lines
Baseline `DomRho8.lean`	30,114	517
Initial pruned numerical proof	21,218	387
Final generic core	19,752	388
Final numerical adapter	2,526	70
Core plus adapter	22,278	458

The combined count includes the new core rather than presenting the adapter alone as the entire proof. Optional examples, drivers, and archived input are separate. The main improvement is the weaker theorem interface and elimination of the deduction calculus; line counts depend on formatting.

The exact numerical-client dependencies were fetched at the pinned commit and freshly compiled. Both the initial numerical reduction and final adapter were checked with the unchanged `KCodeDom.cdom_iff_c222` client. No old project objects were reused for the focused clean check. The main clean Lean closure contains 28 modules, including the generic core, numerical client, optional simulation adapter and subterm example. A separate fresh 13-module closure checks the four-scheme numerical client; the two closures contain 30 distinct project modules. The generic result's printed axiom set is empty; the existing numerical interfaces retain only `propext` and `Quot.sound`. Exact reports are included.

The Rocq abstract core directly ports the finite proof terms. Its carrier, operation, relation and predicate are parameters. Fresh compilation and recursive `coqchk` passed; the exported

unrestricted-row, composition and distribution theorems are closed under the global context. The maintained complete Rocq endpoint has not been switched or rebuilt here.

4 Normalized certificates with balanced lookup

Incoming work separately establishes two refinements. Trailing-zero normalization identifies paths with the same implicit infinite zero extension, reducing the source certificate from 2,240 entries to 1,426. Balanced storage replaces linear list indexing by lookup in a certified binary tree. The present contribution checks their composition.

4.1 Exact refinement replaces a new semantic proof

For a valid certificate tree T, the inherited theorem gives

$$\mathrm{getTree}(T, j) = \mathrm{getc}(\mathrm{flatten}(T), j).$$

The new `BalancedNormalized.v` lifts this equality through branch and entry checks and traversal of the entries:

$$\mathrm{ncheckTree}(p, q, T) = \mathrm{ncheck}(p, q, \mathrm{flatten}(T)).$$

The normalized checker's existing soundness theorem then supplies the equivalence. No additional induction on machine execution and no new restriction on register values is introduced.

The concrete theorem is

$$\forall R, \quad \mathrm{rm_halts}(p215, ([], R)) \longleftrightarrow \mathrm{rm_halts}(dagPerm, ([], R)).$$

The register file R is arbitrary. Bounded segment extraction constructs a finite certificate; it does not bound the execution in this semantic theorem.

The tree constructor is untrusted. Separate kernel-checked statements establish exact flattening, validity, 1,426 entries and height 12. The complete concrete theorem was freshly compiled with Coq 8.18.0 and recursively checked by `coqchk`. The compatibility shim for inherited Rocq 9.1 source is documented and retained.

4.2 Operation counts are not a speedup claim

There is one entry branch and two outgoing branches for each of 1,425 non-halt entries, hence 2,851 lookups. Row 19 is the sole halt entry. Exact counts are

Representation	Visited nodes or cells
Linear list, normalized certificate	2,033,059
Balanced tree, normalized certificate	32,961
Maximum balanced lookup depth	12

This is about 61.7 times less counted lookup work. It is not a corresponding runtime claim. The complete new concrete module took 15.741 seconds in this run, versus 9.238 seconds for the normalized list module. The new module also constructs and proves properties of the tree; these are not repeated controlled benchmarks. The recursive concrete audit took 313.451 seconds, and the full recorded run took 374.868 seconds.

Keep this as a verified optional storage refinement, and measure the production build before preferring it. The normalized list certificate may be the simpler immediate integration choice. Saving one class of evaluator operations does not establish a faster full proof.

5 A reusable halting bridge: benefit and cost

An earlier FM215 proof-compression proposal supplies a generic positive-simulation theorem. We instantiate it at the current `KSim` definitions, replacing the specialized halting proof by one application and preserving the public theorem types.

The reusable principle relates source and target configurations at checkpoints. Each source step corresponds to a positive finite number of target steps, restoring the relation at the next checkpoint. A terminal source configuration leads to target halting after finitely many target steps. The reverse halting implication follows from deterministic execution and strictly positive simulated progress. Positivity prevents an infinite source run from being represented by finitely many target steps and cannot be dropped when replacing bespoke fuel arguments.

The focused integration changes `KSim.lean` from 355 lines and 16,074 bytes to 324 lines and 14,758 bytes. Public type and axiom reports compare identically. However, first use adds 99 inherited generic-theorem lines and 63 adapter lines. Total source grows by 131 lines in this isolated adoption.

This is useful when shared across machines or compiler stages. It is not the leading compression result for a one-machine endpoint. The packet labels it optional, attributes the generic theorem, and records its check separately.

6 Which other 215-state candidate is easier?

The smallest new mathematical obligation and the shortest route to another complete formal endpoint are different criteria.

Candidate	Attraction	Remaining work
Primary K215	Accepted complete proofs; directly benefits from the new core.	Integrate and rebuild the production closure.
CM215	Existing compiled concrete dense/Core equivalence to K215.	Compile the final ZFC adapter with the accepted endpoint closure.
M124 / FM215	Small local marker invariant; existing finite transport checker.	Connect native arbitrary-tape transport to the dense/zipper endpoint.
ZC215 / GC–ZC215	Cleaner scheduler through strict tag-lowering credit.	Physical decoder, primitive, return and startup adapters.

For the quickest additional formal endpoint, CM215 is the pragmatic choice. It retains the source, calculus, scheduler and startup frame; six changed physical rows are handled by a local invariant and two three-step simulation blocks. Its incoming Core bridge is recorded compiled, while its final ZFC adapter explicitly was not compiled with the full endpoint closure. Composition is mathematically short, but the unbuilt closure should not be described as checked.

For the smallest local mathematical argument, M124/FM215 is attractive. Its generic checker packages marker invariants and whole-tape transport once. They are distinct literal machines; keep each table's exact identity and checker instance.

For an independently cleaner source proof, zero-credit variants remove reserve/debt reasoning. Concrete Cantor-code and scheduler properties are already checked in incoming work. Remaining cost lies in the physical adapter rather than the credit inequality alone.

This packet supplies no new complete alternative-machine endpoint. It recommends the least

remaining work from the current inventory and retains primary K215 as the immediate integration target.

7 Searching for 214 states

The new bounded search considers *edge-specific elimination*. To remove a state q, different incoming transitions may choose different replacement write symbols, directions and target states. This is more flexible than identifying q with one globally chosen survivor.

The checker constructs an inductive finite overapproximation of local tape contexts, starting from every inspected tape window at the initial control state and searches for positive bounded joins between old and replacement executions. New entering bits are unconstrained, so the invariant covers every initial tape, including the blank tape. Leaving the inspected window before an exact join is rejected. A replacement cannot reach the removed state or an unresolved patched row before joining. This defines a conservative local search family; lack of a certificate does not prove a replacement globally incorrect.

Seven 215-state tables were examined, with 214 removable labels apiece, giving 1,498 table/label pairs in each run:

Radius / join cap	Replacement rows	Edges with replacements	Full eliminations
5 / 64	1,291,720	4	0
7 / 192	1,299,460	13	0

These are attempted replacement actions, not distinct complete machines, and not an exhaustive search of 214-state tables. Pinned inputs, controls and the exact C++ source are included.

The forced-transfer table admits two incoming-edge shortcuts. The following separate check certifies their simultaneous use and follows the resulting allocation constraints. Neither shortcut alone deletes its former target state.

7.1 A concrete FT215 follow-up

The new table changes native rows $26/0 : 0R134 \mapsto 0R194$ and $26/1 : 1R38 \mapsto 1L114$, leaving all other rows and the source unchanged. It still has 215 working states. An independent local checker computes 971,847 reachable abstract pairs of a control state and a 15-bit window, beginning with all windows at state 0. Sliding a window supplies both possible exterior bits, so this is an inductive overapproximation for every initial tape, including the blank tape. For all 4,736 eligible row 26 windows, positive old/new traces join with identical control, head and tape before any exterior read. No additional modified row is executed in the strict new-trace interior. Exact positive block matching therefore transports halting in both directions. This is an ordinary finite mathematical certificate, not a proof-assistant theorem.

For row 26/0, the next bit is 0 and both paths take two steps: $26 \to 134 \to 17$ and $26 \to 194 \to 17$. For row 26/1, the next two bits are 00; the verified old/new durations are $(3, 5)$ for 1,920 windows and $(5, 7)$ for 128 windows. Fresh symbolic verification independently proves all 6,152 arbitrary-natural primitive cases on each exact table, using the inherited whole-tape induction rules. The original/new rule counts are 1,617,650/1,618,486. The exact blank checkpoint at old step 70,506 is reached at new step 70,526.

The two edits free engine rows 38/0 and 134/0 but introduce obligations at 46/0, 107/1, 114/0, 114/1, 126/0 and 194/0. Thus engine rows increase 144 to 148 and engine states 80 to 82. This makes the two-step 26/0 shortcut a more attractive future starting point than the longer 26/1 detour.

A bounded global allocation retains the current source and its 15 contextual aliases and protects 210 engine, helper and initial-bootstrap rows. Its control reconstructs exactly the 215-state table. Attempts to delete 38 or 134 both fail. Independently of the MILP solver, each attempt exhibits 115 distinct decoder roles whose entire allowed union has only 114 physical slots. The full Hall witness is included. This proves an obstruction only for the stated pinned alias and protected row model; it is not a 214-state impossibility result.

After the edits, the remaining entrances are native row 188/1 to state 38 at source prefix 11, whose left child is CI6, and native row 312/1 to state 134 at prefix 001, whose left child is CD3. Both right children are nonterminal. Native labels are not consecutive: label 312 is a physical state, while the table still has only 215 distinct working labels. Further progress must change the source role structure, introduce another contextual alias, or separately reimplement one of the protected engine/helper obligations. The new four-shape rho8 theorem suggests another emitter language, but its nested implication constructors and branching must be costed in a literal allocation before claiming a state saving.

7.2 Why removing truth does not automatically remove a boot state

The new calculus theorem does not by itself eliminate a physical instruction. In K215, zero also serves as a stack terminator and initial register value. Startup constructs an interpreter frame and enumeration counter; there is no separate physical instruction whose only purpose is to assert $\mathcal{T}(0)$.

In the current coding, the all-zero restricted-ρ_8 row already derives zero. The abstract result broadens permissible encodings, but a different encoding still needs source and tape invariants. Reinterpreting a startup zero without those invariants would not establish a smaller correct machine.

A useful next compiler target is to combine local engine shortcuts with decoder reallocation. A reachable state can have an unneeded symbol row available for a decoder role. The required saving may therefore involve both the engine schedule and decoder assignment. No theorem here guarantees that this will reach 214.

8 A logical obstruction to one recovery strategy

A separate source-reduction proposal restricts atomic vacuity to a cofinal set of indices. One tempting proof strategy is to recover all ordinary atomic-vacuity instances from the remaining logical rules. The following model blocks that logical-only strategy.

Let $e : \mathbb{N} \to \{0, 1\}$ be a valuation. Interpret equality ordinarily, quantifiers by coordinate updates, and implication classically. Set

$$R(a, b)[e] \quad \Longleftrightarrow \quad e(0) = 0 \text{ and } e(a) \neq e(b).$$

The displayed arguments are a, b, but the relation also depends on coordinate zero.

The checked fragment includes ordinary equality/quantifier rows r_1–r_7, unrestricted propositional r_8, generalization, all equality-vacuity instances with the usual distinct-index conditions, the merged equality row, and relation-vacuity only when

$$a + b < c : \qquad R(a, b) \to \forall c\, R(a, b).$$

All these instances are valid. The cofinal restriction makes c different from $0, a, b$, preserving the hidden coordinate.

But

$$R(1, 2) \to \forall 0\, R(1, 2)$$

is false when $e(0) = e(1) = 0$ and $e(2) = 1$: changing coordinate zero to one destroys the relation. Soundness yields a formal nonderivability theorem. `HiddenCoordinate.lean` contains the complete definitions, row-by-row semantics, soundness induction and witness.

This is not a model of the set-theoretic seed bundles. It does not refute equiconsistency of the full restricted calculus or establish that a TM candidate is wrong. It blocks deriving all missing vacuity instances from this logical fragment alone. A successful proof may use the bundles, interpret the weakened calculus differently, or prove equiconsistency without recovering every ordinary first-order axiom.

9 Integration and reproduction

Recommended order:

1. Add the generic implication core and replace `DomRho8.lean` with its compatibility adapter. Update the generator too, so regeneration preserves the new proof.
2. Recheck `KCodeDom.cdom_iff_c222` and the two production K215 endpoint clients. The delivered focused client check is not a substitute for the production dependency-closure gate.
3. Instantiate the checked Rocq abstract core in the maintained numerical lane, then run its full endpoint build and assumption audit.
4. Adopt the incoming normalized source certificate. Treat balanced storage as optional and measure production cost.
5. Use the positive-simulation adapter where shared reuse justifies its additional source. Do not count the smaller `KSim` file alone as net compression.
6. For another endpoint, finish CM215's adapter closure or the M124/FM215 native-to-dense bridge, binding exact table hashes.

The archive includes proof sources, deterministic generation data, patches, pinned minimal dependencies, receipts and logs, and search artifacts. It excludes compiled objects, toolchains, full repository copies and superseded whole incoming archives. The lightweight audit checks hashes, counts and bindings. Proof-assistant replays are separate explicit commands.

9.1 Remaining limits

No new complete 214-state machine is claimed. The generic theorem simplifies a calibration layer. The separate CFour adapter does prove equality of derivability with $C222$, but no source enumerator or tape machine for that presentation has been compiled here. The full production Lean and Rocq endpoint closures were not rebuilt after these edits. The preserved interfaces and focused checks make the replacement concrete, but the integration gate remains necessary.

The next mathematical question is which restricted implication presentations admit a cheaper enumerator. The next compiler question is whether the local engine shortcuts release enough symbol rows to remove a decoder state. This packet provides checked proof cores and concrete local constraints for those questions.

A Trust and provenance

The main implication results are finite proof terms checked by the proof assistants. Search and generators choose substitutions and a dependency order; they introduce no new axioms. The source-checker refinement reuses the inherited register semantics and segment proof. Its finite evaluations are kernel-checked, and recursive `coqchk` audits the concrete theorem.

The generic core retains the baseline Source 149/CD230 provenance and records thirteen replacement steps. Normalized and balanced components retain both incoming notices. Simulation extraction is attributed to the earlier FM215 proposal. Reference machine tables are identified by exact paths and hashes, not state counts alone.

References

[1] ProveIt, K215 candidate and verification index, commit `aa66d0a33f1f8237a348899eb8095b045a502832`.
https://github.com/VladimirReshetnikov/ProveIt/tree/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/candidate-k215

[2] ProveIt, phase-five ordinary certificates and shared semantic lemmas.
https://github.com/VladimirReshetnikov/ProveIt/tree/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase5

[3] ProveIt, baseline `DomRho8.lean` and `KCodeDom.lean`.
https://github.com/VladimirReshetnikov/ProveIt/tree/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Lean/BusyBeaver/ZFC

[4] ProveIt, three additional proof-research packets in the baseline commit.
https://github.com/VladimirReshetnikov/ProveIt/commit/aa66d0a33f1f8237a348899eb8095b045a502832

[5] ProveIt, K215 alternatives at the same pin.
https://github.com/VladimirReshetnikov/ProveIt/tree/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives

[6] Accompanying `K215_Proof_Core_2026-10-21.zip`. Its manifest, local proof receipts, pinned-source records and scripts supply evidence for the new measurements and theorem checks reported here.